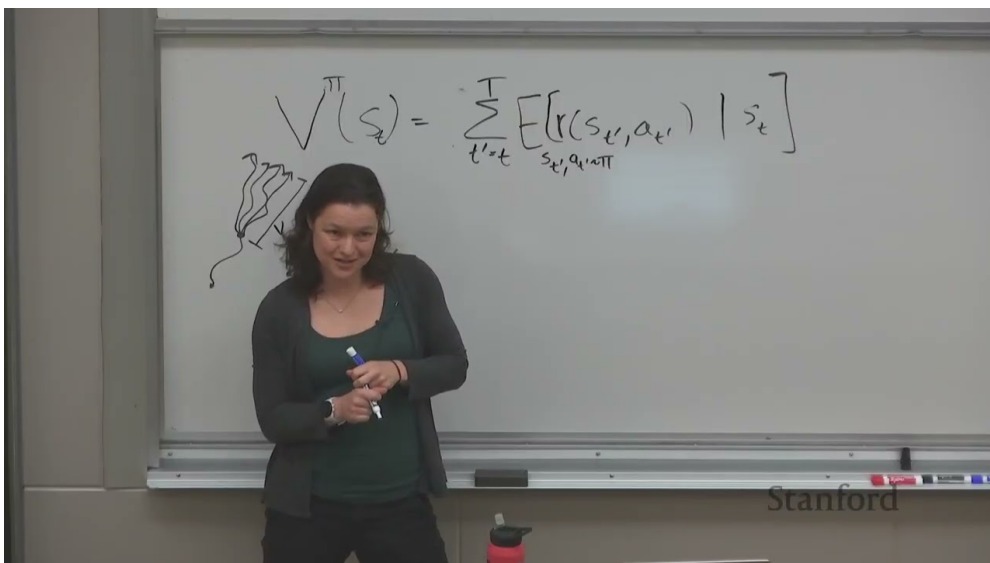


课程笔记

# CS224R Lecture 4: Actor-Critic 方法

基于公开课程资料整理

2025 年春季



视频作者/频道: Stanford Online

发布日期: 2025-04

视频时长: 约 80 分钟

讲者: Chelsea Finn

课程主页: [cs224r.stanford.edu](https://cs224r.stanford.edu)

项目主页: <https://github.com/hqhq1025/ai-course-notes>

# 目录

|                                      |           |
|--------------------------------------|-----------|
| <b>1 回顾：策略梯度的不足</b>                  | <b>3</b>  |
| 1.1 本章小结                             | 3         |
| <b>2 核心对象回顾</b>                      | <b>3</b>  |
| 2.1 本章小结                             | 3         |
| <b>3 Actor-Critic 框架</b>             | <b>3</b>  |
| 3.1 用优势函数改进策略梯度                      | 3         |
| 3.2 如何估计价值函数                         | 3         |
| 3.3 价值函数的拟合方法                        | 4         |
| 3.4 架构与数据流                           | 4         |
| 3.5 Critic 作为 baseline 与正则项          | 5         |
| 3.6 Generalized Advantage Estimation | 5         |
| 3.7 Actor-Critic 完整流程                | 5         |
| 3.8 本章小结                             | 5         |
| <b>4 稳定性、正则与调度</b>                   | <b>6</b>  |
| 4.1 Entropy 与多样性正则                   | 6         |
| 4.2 Trust region 与 clipped objective | 6         |
| 4.3 调度与 target network               | 6         |
| 4.4 本章小结                             | 6         |
| <b>5 Off-Policy Actor-Critic 简介</b>  | <b>7</b>  |
| 5.1 本章小结                             | 7         |
| <b>6 工程实践与调参</b>                     | <b>7</b>  |
| 6.1 Replay buffer 和数据采样策略            | 7         |
| 6.2 Batching 与多步回报的实践                | 7         |
| 6.3 调参与故障排查                          | 8         |
| 6.4 本章小结                             | 8         |
| <b>7 主流 Actor-Critic 族与诊断</b>        | <b>9</b>  |
| 7.1 算法族与设计差异                         | 9         |
| 7.2 训练监控与实验指标                        | 9         |
| 7.3 本章小结                             | 10        |
| <b>8 评测与基准</b>                       | <b>10</b> |
| 8.1 评价指标体系                           | 10        |
| 8.2 Benchmark 例子                     | 10        |
| 8.3 本章小结                             | 10        |

|                                         |           |
|-----------------------------------------|-----------|
| <b>9 奖励塑形与时间信用分配</b>                    | <b>11</b> |
| 9.1 卷积 LSTM 和 reward shaping . . . . .  | 11        |
| 9.2 可微分规划器与 credit assignment . . . . . | 11        |
| 9.3 本章小结 . . . . .                      | 11        |
| <b>10 策略安全与系统化部署</b>                    | <b>12</b> |
| 10.1 Risk-aware fine-tuning . . . . .   | 12        |
| 10.2 在线监控与 rollback . . . . .           | 12        |
| 10.3 本章小结 . . . . .                     | 12        |
| <b>11 超参追踪与日志化</b>                      | <b>13</b> |
| 11.1 Logbook 结构 . . . . .               | 13        |
| 11.2 自动化表格 . . . . .                    | 13        |
| 11.3 本章小结 . . . . .                     | 13        |
| <b>12 实验复现与发布</b>                       | <b>13</b> |
| 12.1 从数据到可部署模型的流水线 . . . . .            | 13        |
| 12.2 版本管理与回滚策略 . . . . .                | 14        |
| 12.3 本章小结 . . . . .                     | 14        |
| <b>13 公式与调度</b>                         | <b>14</b> |
| 13.1 Advantage 与 TD 计算公式组合 . . . . .    | 14        |
| 13.2 Policy loss 与 entropy . . . . .    | 14        |
| 13.3 调度 heuristics 与学习率 . . . . .       | 15        |
| 13.4 本章小结 . . . . .                     | 15        |
| <b>14 实战案例：机器人控制与多智能体</b>               | <b>16</b> |
| 14.1 Robotics 中的 PPO/SAC 选择 . . . . .   | 16        |
| 14.2 多智能体训练中的共享 critic . . . . .        | 16        |
| 14.3 本章小结 . . . . .                     | 17        |
| <b>15 总结与延伸</b>                         | <b>17</b> |
| 15.1 未来方向 . . . . .                     | 17        |
| 15.2 泛化与域迁移 . . . . .                   | 18        |
| 15.3 总结表 . . . . .                      | 18        |
| 15.4 拓展阅读 . . . . .                     | 19        |

## 1 回顾：策略梯度的不足

策略梯度使用采样的 reward-to-go 来估计每个动作的好坏，但这种估计方差很大。

### 策略梯度的两个核心问题

1. **数据利用不充分**：稀疏奖励下，大部分轨迹的奖励为零，梯度信息很少。
2. **高方差**：采样 reward-to-go 是单次采样估计，噪声大。

核心改进思路：能否学习什么是好的、什么是坏的，而不是仅依赖采样估计？

### 1.1 本章小结

策略梯度的主要敌人是高方差与低效利用轨迹，构造一个学习的 critic 去估计优势函数可以显著改善这种现象，并为 Actor-Critic 系统的出现打下基础。

## 2 核心对象回顾

### 价值函数、Q 函数与优势函数

- **价值函数**  $V^\pi(s)$ ：从状态  $s$  出发遵循  $\pi$  的期望未来奖励。
- **Q 函数**  $Q^\pi(s, a)$ ：从  $s$  执行  $a$  后遵循  $\pi$  的期望未来奖励。
- **优势函数**  $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ ：动作  $a$  相对于策略  $\pi$  的平均表现好多少。

关键关系： $V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} [Q^\pi(s, a)]$

### 2.1 本章小结

$V$ 、 $Q$ 、 $A$  三个函数从不同角度描述策略的好坏。优势函数  $A$  特别有用，因为它直接衡量某个动作相对于策略平均水平的好坏。

## 3 Actor-Critic 框架

### 3.1 用优势函数改进策略梯度

用  $A^\pi$  替代 reward-to-go：

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_{i,t} | s_{i,t}) \cdot \hat{A}^\pi(s_{i,t}, a_{i,t})$$

更好的  $\hat{A}$  估计  $\Rightarrow$  更低噪声的梯度。

### 3.2 如何估计价值函数

只需拟合  $\hat{V}^\pi$  即可，因为：

$$\hat{A}^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)$$

这里利用了采样得到的  $s_{t+1}$  来近似期望。

### 3.3 价值函数的拟合方法

**方法 1: Monte Carlo 回归**

$$\min_{\phi} \sum_{s_t \in \mathcal{D}} \left\| \hat{V}_{\phi}^{\pi}(s_t) - \sum_{t'=t}^T r(s_{t'}, a_{t'}) \right\|^2$$

直接用采样的累计奖励作为 target。无偏但高方差。

**方法 2: Bootstrapped / TD 更新**

$$\min_{\phi} \sum_{(s, a, s') \in \mathcal{D}} \left\| \hat{V}_{\phi}^{\pi}(s) - \left( r(s, a) + \gamma \hat{V}_{\phi'}^{\pi}(s') \right) \right\|^2$$

用自身的估计作为 target (bootstrap)。低方差但有偏。

**方法 3: n-step 回报**

结合上述两者，使用  $n$  步实际奖励 + bootstrap:

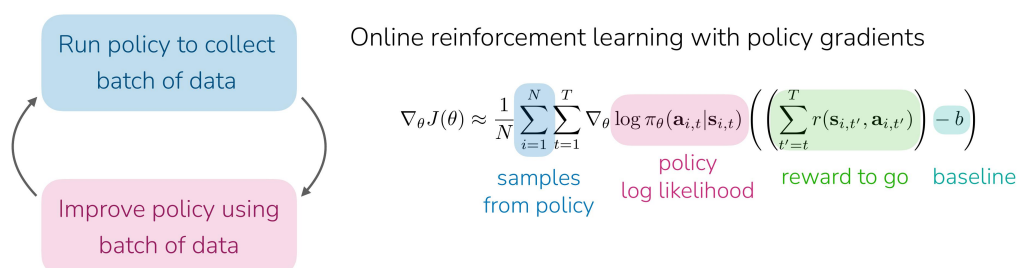
$$y_{i,t} = \sum_{t'=t}^{t+n-1} r(s_{i,t'}, a_{i,t'}) + \gamma^n \hat{V}^{\pi}(s_{i,t+n})$$

#### Bias-Variance Tradeoff

- Monte Carlo: 无偏，高方差。
- TD (1-step bootstrap): 有偏（因为  $\hat{V}$  不精确），低方差。
- n-step: 在两者之间提供可调的平衡。

### 3.4 架构与数据流

#### Recap of Last Time



Do more of the above average stuff, less of the below average stuff.

3

图 1: Lecture slides 上的 Actor-Critic 数据流示意图: 策略 (Actor) 输出动作, critic 提供优势估计反馈梯度。

图中可以看到一个典型的 Actor-Critic 循环：从当前策略生成动作，存入经验池；Critic 使用经验数据拟合价值函数或者 Q 值；Advantage 估计后发送回 Actor 做梯度更新。整个流程是异步的，但细粒度地保持数据一致性（fixed target network、延迟更新）是稳定性的关键。

### 3.5 Critic 作为 baseline 与正则项

Critic 不只是计算优势，它同时充当 baseline，并可通过正则项提升鲁棒性。例如，L2 正则化、value clipping 和 bootstrap target 的 moving average 都是避免 critic drift 的常见做法。

#### critic drift 造成的训练崩溃

Critic 若训练过快，会把价值函数拉到高估区域，导致 advantage 始终为正，Actor 会倾向于某些动作并迅速收敛到劣化策略。控制 critic 学习率和添加 target network 是避免这种“pessimistic collapse”的常用招数。

#### Probabilistic baseline vs learned critic

传统策略梯度用 episode average reward 作为 baseline，Actor-Critic 直接学一个参数化函数，更紧贴当前状态。学习式 critic 的优势在于可以处理长期依赖（如 bootstrapped n-step 回报）并主动修正偏差，提高样本效率。

### 3.6 Generalized Advantage Estimation

GAE 用一个指数衰减核融合多步回报，公式为：

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

其中  $\lambda \in [0, 1]$  控制 bias-variance tradeoff。  $\lambda = 0$  退化为 1-step TD，  $\lambda = 1$  退化为 Monte Carlo。

#### GAE 的调度思路

实践中逐渐 anneal  $\lambda$  从较低值走向 1，可以在训练早期保持低方差，在收敛期逐步吸收长时间依赖信息。结合 target network 和  $\gamma$  的 decay，还能避免 critic 过拟合短期 reward。

### 3.7 Actor-Critic 完整流程

1. 运行策略收集数据
2. 拟合价值函数  $\hat{V}^\pi$
3. 估计优势  $\hat{A}^\pi$
4. 用策略梯度更新策略  $\pi_\theta$
5. 重复

### 3.8 本章小结

Actor-Critic 通过学习价值函数来估计优势，替代了策略梯度中噪声很大的采样 reward-to-go，显著降低了梯度方差。

## 4 稳定性、正则与调度

### 4.1 Entropy 与多样性正则

$$\mathcal{L}_\pi = -\mathbb{E}_{s \sim \mathcal{D}, a \sim \pi} [\hat{A}(s, a) \log \pi(a|s)] + \beta \cdot \mathcal{H}(\pi(\cdot|s))$$

Entropy 项鼓励策略在更新阶段保留一定随机性，从而避免策略迅速收敛到次优动作。这在稀疏奖励或多模态动作空间尤为重要。

#### Entropy regularization 的实际量级

在 PPO / SAC 中常见做法是把 entropy bonus  $\beta$  设为策略 loss 的 0.01 0.1，即保留 1

### 4.2 Trust region 与 clipped objective

PPO 的 clipped objective:

$$L^{CLIP} = \mathbb{E} \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

其中  $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ 。

把 KL 约束或者 clip 机制当作 trust region，可以防止策略步幅过大。讲者提示：在 PPO 里监控  $KL(\pi_{\theta_{\text{old}}} || \pi_\theta)$ ，一旦跳过阈值就退回到上一次 checkpoint。

#### Trust region vs clipping

TRPO 通过精确算梯度约束，计算量大；PPO 则把约束用 clip 近似，计算简单且易于并行，是当前主流首选算法。

### 4.3 调度与 target network

Critic 的 Target Network（如 SAC 的慢速权重）通过 Polyak averaging 让目标值平滑。例如：

$$\theta_{\text{target}} \leftarrow \tau \theta + (1 - \tau) \theta_{\text{target}}$$

较小的  $\tau$  值让 target network 更新缓慢，提高稳定性，但也可能在非平稳环境里反应迟钝。

#### Target network 更新太慢会跟不上策略

如果  $\tau$  设得过小，critic 会一直追随已经“过时”的 target，导致引导 Actor 估计过于乐观，对新样本适应慢。通常把  $\tau$  设在 0.005 0.05 之间。

### 4.4 本章小结

稳定性技巧（entropy、clip objective、target network）是 Actor-Critic 能否快速收敛的关键。监控 KL 与 advantage 的分布可以及时发现出界更新。

## 5 Off-Policy Actor-Critic 简介

当结合 importance weights 时，Actor-Critic 可以复用旧数据进行多步梯度更新。

$$\nabla_{\theta'} J(\theta') \approx \sum_{t,i} \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_{\theta}(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \hat{A}^{\pi_{\theta}}(s_{i,t}, a_{i,t})$$

但 importance weights 在新旧策略差异大时不稳定。下一讲将介绍如何用 KL 约束和 clipping 来解决这个问题。

### 5.1 本章小结

Off-policy actor-critic 通过 importance sampling 复用旧数据，但需要限制策略更新的步幅以保持稳定性。

## 6 工程实践与调参

### 6.1 Replay buffer 和数据采样策略

现代 off-policy Actor-Critic 模型往往使用优先级 replay buffer、frame stacking 和 multi-environment rollout。过往讲义中也反复强调：数据采样策略直接决定了 critic 的训练信号。

#### Prioritized replay 的小技巧

把 TD error 的绝对值作为 priority，在 sampling 时增加概率，并在 loss 中乘以 importance weight correction，确保 update 不会偏向高 error 样本。

### 6.2 Batching 与多步回报的实践

批量大小、采样频率与 n-step 的组合是基础调参方向。可以通过下表把常用组合总结出来。

| 配置项              | 常见取值                   | 训练影响                                  |
|------------------|------------------------|---------------------------------------|
| Batch size       | 256–4096 transitions   | 越大 variance 越低，但硬件吞吐是瓶颈               |
| n-step           | 1–5                    | 大步数偏差小但有 drift，常用 $\lambda$ mix       |
| Update frequency | 1 update per 1–4 steps | 更少 update 减少 off-policy bias          |
| Replay ratio     | 1–4                    | >1 表示每次采样后更新多次，各步 learning rate 需对应减小 |

表 1: 实践中常调的超参组合

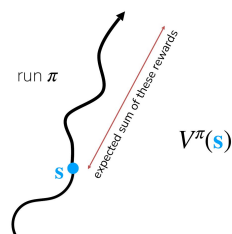


## 6.3 调参与故障排查

### Estimating $V^\pi$

Version 1: Monte Carlo estimation

$$V^\pi(s_t) \approx \sum_{t'=t}^T r(s_{t'}, \mathbf{a}_{t'}) \quad \leftarrow \text{original single sample estimate}$$



15

图 2: Lecture 4 中展示的训练流程图：多 worker 拉取数据、critic 拟合目标、Actor 更新策略。

当训练出现 divergence, 先看 three signals: critic loss、KL、reward curve。此外, 常见的 failure mode 有:

- Critic overestimate: 回归失灵导致 advantage 恒正;
- Replay buffer stale: 环境改变但 buffer 中数据仍旧;
- Exploration collapse: entropy 快速降为 0。

#### 出现 divergence 时的快速复位流程

先把策略退回上一个 checkpoint, 关闭 entropy decay, 减小 learning rate, 再逐步恢复。不要一口气加大数据量, 否则会加速灾难。

## 6.4 本章小结

实践中, 数据采样、batching 和超参调度构成 Actor-Critic 稳定训练的系统性问题。监控 replay ratio、entropy 和 critic loss 是调参的第一手数据。

## 7 主流 Actor-Critic 族与诊断

### 7.1 算法族与设计差异

| 算法       | Critic 类型            | 策略更新                         | 适合场景          |
|----------|----------------------|------------------------------|---------------|
| PPO      | V 的 soft target      | clipped surrogate            | 模型同步训练、CPU 集群 |
| A2C/A3C  | 单步 V + entropy       | 过采样与不同 worker                | 齐心并行、多实例      |
| SAC      | 双 Q + entropy max    | off-policy replay buffer     | 连续动作、需高样本效率   |
| DDPG/TD3 | Deterministic policy | deterministic actor gradient | 高维连续控制        |

表 2: 不同 Actor-Critic 变体的核心差异

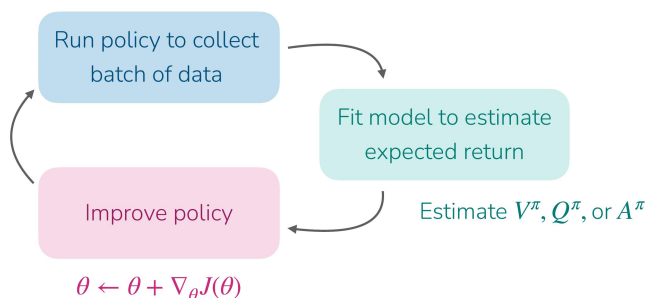
#### 从 Lecture slide 中抽象出来的设计维度

讲者强调的差异主要来自三个维度: 更新频率 (on-policy vs off-policy), 目标函数约束 (clipping / KL / trust region), 以及是否带 entropy 或自动调节 exploration。

### Online RL Outline

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) A^{\pi}(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

First: Initialize the policy (randomly, with imitation learning, with heuristics)



12

图 3: Lecture slides 中关于 PPO 与 SAC 的对比图, 强调了 on-policy 的稳定性与 off-policy 的样本效率互补。

### 7.2 训练监控与实验指标

- **KL divergence**: 监控更新是否突破 trust region;
- **Advantage 分布**: 过宽代表 high variance, 过窄代表 bias 大;
- **Critic loss 曲线**: 暴涨意味着 learning rate 太大或目标不稳定;

- **Entropy**: 下降太快说明 exploration 落入 determinism。

### 指标之间的 tradeoff

降低 critic loss 并不总是好事——只要 advantage 估计偏差大, Actor 就会收到错误信号。稳定训练需要同时看 loss、KL 和 policy entropy 的趋势。

## 7.3 本章小结

主流 Actor-Critic 族在 trust region、entropy 和 off-policy buffer 上各有侧重; 实战中需要观测多条曲线并配合 early stopping 或 checkpoint rollback。

# 8 评测与基准

## 8.1 评价指标体系

- **Sample efficiency**: 每次 update 平均消耗多少 transitions;
- **Reward saturation**: reward 曲线 plateau 是否稳定;
- **Entropy decay**: entropy 快速下降表示 exploration collapse;
- **Policy divergence**: KL 及梯度方向是否与 critic signal 保持一致。

### 审慎设定评测窗口

测试每个 checkpoint 时要使用固定随机种子和 rollout steps, 防止 single-run high reward 误导结论。建议把 best-of- $N$  分数与均值、标准差一起上报。

## 8.2 Benchmark 例子

- **MuJoCo locomotion**: PPO 作为 baseline, SAC 提供更高样本效率;
- **Atari**: A3C/A2C 展示了批量并行能显著缩短收敛;
- **Robotics sim2real**: 严格控制 KL、safety filter 与 entropy;
- **多智能体 StarCraft**: centralized critic + decentralized actor。

### Benchmark 时常犯的错误

只在 single random seed 下复现 best score, 会掩盖 policy collapse 和 training instability。建议至少跑 3-5 条种子, 并把 KL 轨迹与 reward curve 一起画。

## 8.3 本章小结

评测不只是看 final reward, 还要监控 entropy、KL、critic loss 与 sampling ratio, 才能判断 Actor-Critic 是否真正稳定。

## 9 奖励塑形与时间信用分配

### 9.1 卷积 LSTM 和 reward shaping

#### The plan for today

##### Actor critic methods

1. Improving policy gradients
2. How to estimate the value of a policy ("policy evaluation")
  - a. Sample & directly supervise ("Monte Carlo estimation")
  - b. Use your own estimate ("bootstrapping", "temporal difference learning")
  - c. Something in-between? ("N-step returns")
3. **Off-policy actor-critic**
  - a. Importance weights & constraining step size
  - b. Full off-policy version with replay buffers

##### Key learning goals:

- How to estimate how good a state and action is for a policy
- How to use those estimates to form a better RL algorithm

24

图 4: Lecture slide 24 描绘的 reward shaping 示例: 在 multi-stage task 中用 intermediate reward guidance 缩短训练时间。

课程强调在 multi-stage/ hierarchical task 中提供 intermediate reward 能显著加速 critic 收敛。做法是把 reward 分解为 *completion reward* + *auxiliary reward*, 并用 dynamic weighting 控制 gradient magnitudes。

#### Reward shaping 的 guardrails

辅助 reward 必须保持 potential-based (即不改变 optimal policy), 否则会鼓励 agent 偏离真正 objective。通常引入 shaping 函数  $F(s, a, s')$  并确保  $\sum_t \gamma^t F(s_t, a_t, s_{t+1})$  的总和为零。

### 9.2 可微分规划器与 credit assignment

讲者提到一个实践 trick: 在复杂控制中用 differentiable planner 估算未来 trajectory, 再把 planner 的 soft value 作为 critic 监督信号。这样能显著减少 long-term credit assignment 的 drift。

#### Soft planner 需要额外计算

引入 differentiable planning 会增加每步推理开销, 必须确保该 planner 能在 low latency 环境里运行, 否则会毁掉 real-time control。

### 9.3 本章小结

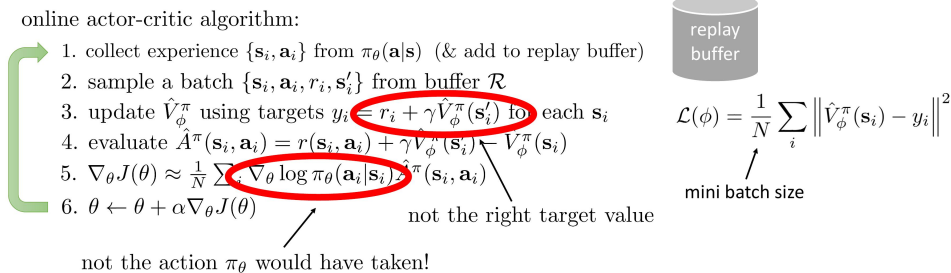
Reward shaping + planner 信号是处理 multi-stage task 和 long-horizon credit assignment 的现实解决方案, 但要确保 shaping 保持 potential-based, 同时不抢占 inference budget。

## 10 策略安全与系统化部署

### 10.1 Risk-aware fine-tuning

#### Off-Policy Actor-Critic Methods

Version 2: Replay buffers



The algorithm is currently broken 🤔

Slide adapted from Sergey Levine

30

图 5: Lecture slide 30 总结的风险感知 pipeline: 先用 conservative critic 约束值, 再在部署前做 safety check。

课程提到一个安全层: 在 actor update 之前检查 critic 输出, 如果当前策略值突破 pre-defined threshold 就回滚上一步。这类似 trust region 的 analog, 但更偏向 deployment guard。

#### Conservative critic 的建议

保存多组 critic checkpoints, 部署时选取 conservative 一组。这样即便 newest critic overestimates, actor 也不会被过强信号误导。

### 10.2 在线监控与 rollback

在生产系统中, 常见做法是把 KL、critic loss 和 entropy 平均值推到 dashboard。若任一指标连续 3 次超限, 就自动触发 rollback 和 entropy bonus 重启。

#### 生产系统的经验

LLM 或 RL 系统里, policy divergence 的 early warning 信号非常重要。课程建议用 3-day rolling window 统计 KL, 如果超过 2x historic std, 就暂停 actor 更新, 让 critic catch up。

### 10.3 本章小结

安全部署要求 actor-critic 系统在更新与 inference 之间形成防火墙: conservative critic, guard rails, 自动 rollback 与持续监控。

## 11 超参追踪与日志化

### 11.1 Logbook 结构

课程建议为每次实验维护一份 logbook，记录以下字段：

- **Learner IDs**: Actor、Critic、Target network 的 LR;
- **Replay ratio**: 每个 update 消耗的 transitions;
- **Entropy schedule**: 初始值 + decay 速率;
- **Benchmark metrics**: KL、critic loss、reward mean。

#### 为什么要 log mutation

比较不同 seed 时，往往只有超参数略微偏差。记录完整 logbook 能让你快速复制 good run，也能快速排查 catastrophic divergence。

### 11.2 自动化表格

| 字段           | 推荐工具                | 目的                        |
|--------------|---------------------|---------------------------|
| KL curve     | TensorBoard / WandB | 监控是否突破 trust region       |
| Entropy      | WandB custom scalar | 追踪 exploration collapse   |
| Critic loss  | CSV + daily summary | 发现 critic drift 前兆        |
| Replay ratio | elastic log         | 确保 sample efficiency 不被忘记 |

表 3: Hyperparameter logging 的具体实践

### 11.3 本章小结

超参日志化将实验从“跑几个 seed”升级为“可复现的工程流程”，对齐团队协作与 performance regression 检查。

## 12 实验复现与发布

### 12.1 从数据到可部署模型的流水线

复现实验包含三道关卡：数据收集、训练循环、部署 guard。其中每一关都要持续监控 metrics (KL、entropy、critic loss)，并维护实验日志来比较不同 run。

- **数据**: 使用 deterministic rollout 保存 state/action/reward/truncation;
- **训练**: 搭建 actor/critic/stage scheduler、自动保存 best checkpoint;
- **部署**: 将 conservative critic 作为 inference-time guard，保留 fallback policy。

### 复现流水线的三步守则

1) 先确保 replay buffer 能被序列化; 2) 保持 actor update 与 critic update 在同一 config; 3) 在 guard stage 触发 rollback 时记录当时的 KL 和 entropy。

## 12.2 版本管理与回滚策略

为了快速回滚, 课程建议用 git tag 记录每个 checkpoint 对应的 config (learning rate、lambda、entropy bonus)。若 new run 出现 high KL, 就 revert 到 tag, 避免误合并。

### 回滚策略的误区

不要在 rollback 过程中重置 replay buffer, 否则会丢失之前的 critic signal; 只需回到 checkpoint, 保持 buffer state, 重新 warm up entropy。

## 12.3 本章小结

实验复现要把训练、日志、回滚打包成一个可执行 pipeline, 这样才能在多个团队 copy/paste 同一个好结果。

## 13 公式与调度

### 13.1 Advantage 与 TD 计算公式组合

$$\begin{aligned}\delta_t &= r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \\ \hat{A}_t^\lambda &= \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \\ V_\phi(s_t) &\leftarrow V_\phi(s_t) + \alpha_\phi \delta_t \\ \pi_\theta &\leftarrow \pi_\theta + \alpha_\theta \hat{A}_t^\lambda \nabla_\theta \log \pi_\theta(a_t|s_t)\end{aligned}$$

这组公式展示了 actor/critic 之间的信息流: critic 通过 TD error 更新自身, actor 则把预估 advantage 乘以 log probability 做梯度。 $\lambda$  的调度会显著影响 bias-variance tradeoff, 建议在 architecture log 中记录每次变化。

### 13.2 Policy loss 与 entropy

$$\begin{aligned}\mathcal{L}_\pi &= -\mathbb{E} \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] - \beta \mathcal{H}(\pi) \\ \mathcal{H}(\pi) &= - \sum_a \pi(a|s) \log \pi(a|s)\end{aligned}$$

### Clip objective 的稳定策略

The clip objective minimizes whichever term is smaller between the unclipped ratio and the clipped ratio, so when  $r_t(\theta)$  leaves the trust region it safely falls back to the clipped variant. Coupling this with the entropy bonus keeps the early-stage exploration from collapsing too quickly.

## 13.3 调度 heuristics 与学习率

### Aside: discount factors

$$y_{i,t} \approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1})$$

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$

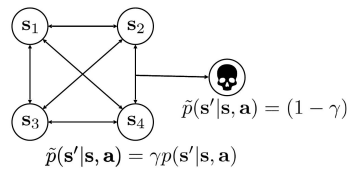
what if  $T$  (episode length) is  $\infty$ ?

$\hat{V}_\phi^\pi$  can get infinitely large in many cases

simple trick: better to get rewards sooner than later  $\gamma$  changes the MDP:

$$y_{i,t} \approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1})$$

discount factor  $\gamma \in [0, 1]$  (0.99 works well)



Slide adapted from Sergey Levine

21

图 6: Slide 展示的 learning rate 和 entropy 调度策略, 鼓励在训练中期逐步衰减 entropy bonus。

常见的调度方案:

- Learning rate anneal: 从  $3e-4$  归零到  $1e-5$ , 用 cosine decay 保持平滑。
- $\lambda$  调度: early stage 用 0.95, 收敛时渐近到 0.99。
- Clipping eps: 当 KL 超过阈值时减小  $\epsilon$  并再 warm up。

### 调度过快的危险

如果 entropy bonus 迅速衰减到 0, 策略很容易陷入 deterministic 的死胡同, 特别是在 reward 稀疏的环境。建议保留一个 floor, 最低也要留 0.01 的 entropy。

## 13.4 本章小结

公式与调度提供了 Actor-Critic 的数学骨架。从 advantage 到 policy loss, 每一项都需要被记录、可热替换, 并在调度 log 中对应具体指标。



## 14 实战案例：机器人控制与多智能体

### 14.1 Robotics 中的 PPO/SAC 选择

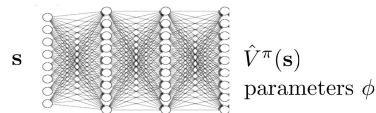
#### Estimating $V^\pi$

Version 2: Bootstrapping

Monte Carlo target:  $y_{i,t} = \sum_{t'=t}^T r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'})$

ideal target:  $y_{i,t} = \sum_{t'=t}^T E_{\pi_\theta} [r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) | \mathbf{s}_{i,t}] \approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \sum_{t'=t+1}^T E_{\pi_\theta} [r(\mathbf{s}_{i,t'}, \mathbf{a}_{i,t'}) | \mathbf{s}_{i,t+1}]$

$$\approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + V^\pi(\mathbf{s}_{i,t+1}) \approx r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \underbrace{\hat{V}_\phi^\pi(\mathbf{s}_{i,t+1})}_{\text{directly use previous fitted value function!}}$$



**Step 1:** Aggregate dataset of “bootstrapped” estimates:

training data:  $\left\{ \underbrace{\left( \mathbf{s}_{i,t}, r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \hat{V}_\phi^\pi(\mathbf{s}_{i,t+1}) \right)}_{y_{i,t}} \right\} \quad \leftarrow \text{update labels every gradient update!}$

**Step 2:** Supervised learning to fit estimated value function

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_\phi^\pi(\mathbf{s}_i) - y_i \right\|^2$$

Also referred to as a form of **temporal difference learning**

18

图 7: Lecture slide 18 展示的 robotics benchmark: PPO 用于稳定 on-policy 控制, SAC 擅长高维连续动作。

在现实机器人控制任务中, 训练通常面临采样限制与高维动作空间。PPO 提供的 trust-region + clipping 架构便于在真实设备上限制步幅, 而 SAC 的 entropy-optimal off-policy buffer 让拟合速度更快。

#### 现场调度的建议

在 robot arm / locomotion 任务中常用策略是: 用短 horizon 的 PPO trial 快速验证 reward 结构; 将最有 promise 的设定迁移到 SAC, 配合 soft target 和 automatic entropy tuning。PPO 结果更好解释、SAC 更节省样本。

### 14.2 多智能体训练中的共享 critic

当多个 agent 共享环境时, common critic 或 centralized value 是提高稳定性的关键。通常做法是训练一个 joint critic  $Q(\mathbf{s}, \mathbf{a})$ , 再把 actor 分段更新:

$$\hat{A}_i = Q(\mathbf{s}, \mathbf{a}) - V(\mathbf{s})$$

Agent 只观察自身 action, 但 critic 看到全局状态, 能够更准确判别局部决策的价值。

#### 共享 critic 的风险

如果 critic 过度依赖 global information, 或没有合适的 value decomposition, 单个 agent 的 gradient 会被全局 noise 掩盖。解决方案包括 information bottleneck、agent-specific mask 与 periodic critic reset。

### 14.3 本章小结

PPO 与 SAC 在智能体训练中扮演不同角色：PPO 当作安全策略，SAC 管理高频采样；多智能体场景则需共享 critic 并注意信息泄露。

## 15 总结与延伸

1. Actor-Critic 方法通过学习价值函数来估计优势，降低了策略梯度的方差。
2. 价值函数可以用 Monte Carlo、TD、或 n-step 方法拟合，各有 bias-variance 权衡。
3. Off-policy 版本通过 importance weights 复用旧数据，提高数据效率。
4. Actor-Critic 是 PPO、SAC 等现代 RL 算法的基础。

### 15.1 未来方向

- **Self-supervised critic**: 利用自监督 representation 提升 value estimation 的 generalization;
- **Hierarchical actor-critic**: 把 policy 分层到五百个动作，从高层决策到低层执行;
- **Co-training actor-critic with world models**: 用 dynamics model 预测 next state, 从而让 critic 有 richer context;
- **Adaptive compute**: 让硬件动态分配更多计算给 reward 变化剧烈的样本。

#### 未来方向的潜在风险

新架构往往牺牲稳定性以换取更高的 score, 务必在 pilot run 中把 KL 和 critic loss 作为 guard rails, 不要盲目放开 entropy。

## 15.2 泛化与域迁移

### Off-Policy Actor-Critic Methods

So far:

- use one batch of policy data for one gradient step (fully on-policy)
- use one batch of policy data for multiple gradient steps (starting to be off-policy)

Can we be even more off-policy?

Can we reuse data from previous batches, i.e. all of the past trial-and-error data?

Key ideas:

- maintain a buffer of all past data “replay buffer”
- adjust equations to remove on-policy assumptions

28

图 8: Lecture slide 28 展示迁移学习设置: Actor-Critic 在 source task 训练后, 在 target task 使用 few-shot fine-tuning。

课程提出的迁移策略是: 先在 source environment 用 PPO/SAC 训练出 policy, 再把 critic 固定, 用 small amount of target data fine-tune actor 的 logits。这样避免 target reward 太稀疏导致 critic drift。

#### 迁移中的分布偏移

迁移时要注意 observation/ action space 的 shift; 可以在 replay buffer 里混入 source sample 并用 importance weight correction, 逐步让 actor 适应 new dynamics。

## 15.3 总结表

| 主题            | 关键措施                         | 风险点               | 实践建议                          |
|---------------|------------------------------|-------------------|-------------------------------|
| Advantage 估计  | Critic 学习 $V$ , 使用 GAE       | 偏差或 high variance | 小步更新 + annealed $\lambda$     |
| Entropy 正则    | 加入 $\mathcal{H}(\pi)$ bonus  | randomness 过大     | 用早期高 entropy、后期 decay         |
| Trust region  | PPO clip / KL target         | step 攀高           | 监控 KL、设回退机制                   |
| Off-policy 利用 | Importance sampling + replay | weights 崩溃        | 用 clipping + target smoothing |

表 4: Lecture 4 中提出的核心操作与注意事项

## 15.4 拓展阅读

- Mnih et al., *Asynchronous Methods for Deep Reinforcement Learning* (A3C).
- Schulman et al., *High-Dimensional Continuous Control Using Generalized Advantage Estimation* (GAE).
- Schulman et al., *Proximal Policy Optimization Algorithms* (PPO).
- Haarnoja et al., *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning* (SAC).
- Lillicrap et al., *Continuous control with deep reinforcement learning* (DDPG).